

Payzaapi API

Accept M-Pesa, mobile money, card, bank and crypto payments in 14 African currencies with one REST API. Then pay out, refund, bill on a schedule and run your POS from the same integration.



Quickstart

Take a test payment in about five minutes.



Accept payments

M-Pesa prompts, hosted checkout and crypto.



Webhooks

Get notified the moment a payment settles.

How a payment works

- 1 Your server calls `POST /api/v1/pay` with the amount, currency and customer.
- 2 For KES we send an M-Pesa prompt to the customer's phone. For every other case you redirect the customer to the `payment_url` we return.
- 3 The customer approves the payment on their phone or on our checkout page.
- 4 We send a signed webhook to your `callback_url`, and you fulfil the order.

Base URL

All endpoints live under `https://payzaapi.co.ke/api/v1`. Requests and responses are JSON, over HTTPS only.

Quickstart

- 1 **Create an account** at payzaapi.co.ke and verify your email.
- 2 **Get test keys** from [Dashboard → API keys](#). You get a public key (`pk_test_...`) and a secret key (`sk_test_...`).
- 3 **Create a test payment** with the request on the right. In test mode it succeeds instantly and your `callback_url` receives a real, signed webhook.
- 4 **Handle the webhook** by [verifying its signature](#) and marking the order as paid.
- 5 **Go live** by swapping in your `pk_live_` and `sk_live_` keys. Nothing else changes.

🔔 Keep your keys in environment variables such as `PAYZA_PUBLIC_KEY` and `PAYZA_SECRET_KEY`, which is what every example on this page reads.

POST /api/v1/pay

cURL

PHP

Node.js

Python

Copy

```
curl -X POST https://payzaapi.co.ke/api/v1/pay \
-H "X-Public-Key: pk_test_xxxxxxx" \
-H "X-Secret-Key: sk_test_xxxxxxx" \
-H "Content-Type: application/json" \
-d '{
  "amount": 1000,
  "currency": "KES",
  "reference": "ORDER_1001",
  "customer": {
    "email": "jane@example.com",
    "name": "Jane Wanjiku",
    "phone": "254712345678"
  },
}
```

```
"callback_url": "https://yourshop.co.ke/webhooks/payza"
}'
```

Authentication

Authenticate every request with your key pair in two headers. Create, rotate and revoke keys in [Dashboard → API keys](#).

Headers

X-Public-Key header **Required**

Your public key, `pk_test_...` or `pk_live_...`.

X-Secret-Key header **Required**

Your secret key, `sk_test_...` or `sk_live_...`.



Call the API from your server only. Your secret key can move money out of your wallet. Never put it in a website, mobile app or public repository. If it leaks, revoke it in the dashboard straight away.

Response 401 Unauthorized

Copy

```
{
  "success": false,
  "message": "Invalid API credentials."
}
```

Test and live mode

Test and live use the same URLs. The key prefix decides the mode.

Mode	Keys	What happens
Test	<code>pk_test_</code> <code>sk_test_</code>	No money moves. Payments succeed instantly, and your <code>callback_url</code> gets a real signed <code>payment.success</code> webhook with <code>"gateway": "test"</code> . Verify and the webhook log work as in live.
Live	<code>pk_live_</code> <code>sk_live_</code>	Real M-Pesa prompts, mobile money, card, bank and crypto payments. Settled funds land in your wallet.

- 🕒 Test mode only simulates success. To exercise your failure handling, POST a sample `payment.failed` body (see [payload](#)) to your own endpoint, signed with your webhook secret.

Errors

Every response includes `success`. When it's `false`, `message` explains what went wrong in plain language you can log or show. Use the HTTP status to decide what to do.

Status	Meaning	What to do
<code>400</code> <code>422</code>	A field is missing or invalid	Fix the request using <code>message</code>
<code>401</code>	Keys missing or invalid	Check both headers and the key mode
<code>403</code>	Account suspended, or currency not enabled	See <code>message</code> , or contact support
<code>404</code>	Not found on your account	Check the reference or ID
<code>405</code>	Wrong HTTP method	Use the method shown in these docs
<code>409</code>	Conflict, such as a reused <code>reference</code>	Use a new unique value

429	Rate limit reached	Wait for <code>Retry-After</code> seconds
500	Temporary problem on our side or the payment network	Retry with backoff, then verify
502		

Response 409 Conflict Copy

```
{
  "success": false,
  "message": "reference already exists. Use a unique value."
}
```

Response 429 Too Many Requests Copy

```
{
  "success": false,
  "message": "Rate limit exceeded. You may make 100 requests per 60 seconds. Retry a"
}
```

Rate limits and pagination

Each API key can make **100 requests per 60 seconds**. Past that you get `429` with a `Retry-After` header. Rely on webhooks instead of polling to stay well under the limit.

List endpoints are paginated. Pass `page` and `per_page` in the query string, and read the `pagination` object in the response.

Query parameters

`page` integer Optional

Page number, starting at 1. Default `1`.

`per_page` integer Optional

Results per page. Default `20`, maximum `100`.

Response 200 OK

Copy

```
{
  "success": true,
  "payouts": [
    ""
  ],
  "pagination": {
    "page": 1,
    "per_page": 20,
    "total": 57,
    "total_pages": 3
  }
}
```

Create a payment

POST `/api/v1/pay`

Starts a payment and returns its `reference` and a `payment_url`. What happens next depends on the currency:

- **KES:** an M-Pesa prompt goes to `customer.phone` straight away. Set `stk_push` to `false` to let the customer choose Airtel Money, Telkom, Equitel or card on our [hosted checkout](#) instead.
- **All other currencies:** redirect the customer to `payment_url`.

A payment stays payable for 30 minutes (see `expires_at`).

Body parameters

`amount` `number` **Required**

Amount in the main unit, e.g. `1000` for KES 1,000. Minimums: KES 1, NGN 100, GHS 1, TZS 500, XOF 100, USD 0.50.

`currency` string **Optional**

Three-letter code, default `KES`. See [currencies](#). Charging in a currency other than your account's home currency needs a multi-currency subscription, or you get `403`.

`customer.email` string **Required**

Customer's email address.

`customer.name` string **Optional**

Customer's name. Shown on receipts and in your dashboard.

`customer.phone` string **Required for KES**

Kenyan mobile number, as `254712345678` or `0712345678`. With an M-Pesa prompt it must be a Safaricom number (07xx or 01xx).

`callback_url` string **Required**

HTTPS URL on your server that receives the [webhook](#).

`reference` string **Optional**

Your own unique ID for this payment, such as an order number. Reusing one returns `409`. Generated for you if omitted.

`stk_push` boolean **Optional**

KES only. Default `true` sends the M-Pesa prompt. `false` skips it so the customer picks a network on `payment_url`.

`redirect_url` string **Optional**

Where hosted checkout sends the customer after paying.

`cancel_url` string **Optional**

Where hosted checkout sends the customer if they cancel.

`description` string **Optional**

Shown to the customer and on your statement.

`metadata` object **Optional**

Any key-value data you want back. It is returned unchanged in every webhook for this payment.

Response fields

`data.reference` string

Store this. You'll use it to [verify](#) and to match webhooks.

`data.payment_url` string

Hosted checkout page for this payment. Always returned, including for KES M-Pesa prompts.

`data.crypto_payment_url` string

Sends the customer straight to [crypto checkout](#).

`data.fee` `data.net_amount` number

Our fee, and what reaches your wallet.

`data.status` string

`pending` in live mode until the customer pays. `success` immediately in test mode.

`data.stk_sent` boolean or null

KES only: whether the M-Pesa prompt was sent.

POST /api/v1/pay

[cURL](#)

[PHP](#)

[Node.js](#)

[Python](#)

[Copy](#)

```
curl -X POST https://payzaapi.co.ke/api/v1/pay \
-H "X-Public-Key: pk_test_XXXXXXX" \
-H "X-Secret-Key: sk_test_XXXXXXX" \
-H "Content-Type: application/json" \
-d '{
  "amount": 2450,
  "currency": "KES",
  "reference": "ORDER_1001",
  "customer": {
    "email": "jane@example.com",
    "name": "Jane Wanjiku",
    "phone": "254712345678"
  },
  "callback_url": "https://yourshop.co.ke/webhooks/payza",
  "metadata": {
    "order_id": "1001"
  }
}'
```

Response 200 OK

Copy

```
{
  "success": true,
  "message": "M-Pesa STK push sent to 254712345678. Customer must enter their PIN to",
  "data": {
    "reference": "ORDER_1001",
    "payment_url": "https://payzaapi.co.ke/payment/checkout?ref=ORDER_1001",
    "crypto_payment_url": "https://payzaapi.co.ke/payment/crypto-checkout?type=api",
    "amount": 2450,
    "fee": 36.75,
    "net_amount": 2413.25,
    "currency": "KES",
    "gateway": "payzaapi",
    "actual_gateway": "mobile_money",
    "phone": "254712345678",
    "status": "pending",
    "environment": "live",
    "stk_sent": true,
    "stk_push": true,
    "expires_at": "2026-09-24T10:30:00+03:00",
    "accepted_methods": [
      "mpesa",
      "airtel",
      "bank_transfer",
      "card",
      "crypto"
    ]
  }
}
```

Hosted checkout

Our checkout page handles every method for you: mobile money in each country, card, bank transfer and crypto. Use it for all non-KES payments, and for KES when your customer might not be on Safaricom.

- 1 Create the payment with `redirect_url` and `cancel_url` (and `"stk_push": false` for KES).

- 2 Redirect the customer's browser to `data.payment_url`.
- 3 They pick a method and pay. We send them back to your `redirect_url`.
- 4 Fulfil the order from the [webhook](#), not from the redirect. A customer can close the tab before being redirected.

🕒 On your `redirect_url` page, call [verify](#) with the reference if you want to show "Payment received" immediately.

POST /api/v1/pay

cURL

PHP

Node.js

Python

Copy

```
curl -X POST https://payzaapi.co.ke/api/v1/pay \  
-H "X-Public-Key: pk_test_XXXXXXX" \  
-H "X-Secret-Key: sk_test_XXXXXXX" \  
-H "Content-Type: application/json" \  
-d '{  
  "amount": 1500,  
  "currency": "KES",  
  "stk_push": false,  
  "reference": "ORDER_1002",  
  "customer": {  
    "email": "otieno@example.com",  
    "name": "Otieno Ouma",  
    "phone": "0733123456"  
  },  
  "callback_url": "https://yourshop.co.ke/webhooks/payza",  
  "redirect_url": "https://yourshop.co.ke/orders/1002/thanks",  
  "cancel_url": "https://yourshop.co.ke/cart"  
}'
```

Verify a payment

GET /api/v1/verify/{reference}

Returns the current status of a payment. In live mode, if the payment is still pending we check with the payment network before answering, so the result is as fresh as

possible.

Use it on your thank-you page, and as a safety net if a webhook hasn't arrived after a few minutes. It's also the right way to confirm an amount before shipping high-value goods.

Path parameters

reference string **Required**

The payment reference, in the URL path.

Statuses

pending	Waiting for the customer
success	Paid. <code>paid_at</code> is set
failed	Declined, timed out or wrong PIN
cancelled	The customer cancelled

GET /api/v1/verify/ORDER_1001 **cURL** PHP Node.js Python Copy

```
curl "https://payzaapi.co.ke/api/v1/verify/ORDER_1001" \
-H "X-Public-Key: pk_test_XXXXXXX" \
-H "X-Secret-Key: sk_test_XXXXXXX"
```

Response 200 OK Copy

```
{
  "success": true,
  "data": {
    "reference": "ORDER_1001",
    "status": "success",
    "amount": 2450,
    "fee": 36.75,
    "net_amount": 2413.25,
    "currency": "KES",
    "gateway": "payzaapi",
```

```
    "actual_gateway": "mobile_money",
    "customer_email": "jane@example.com",
    "customer_name": "Jane Wanjiku",
    "description": "Order 1001",
    "paid_at": "2026-09-24 10:02:41",
    "created_at": "2026-09-24 10:00:05"
  }
}
```

Crypto payments

Every payment includes a `crypto_payment_url`. There's nothing extra to integrate. Redirect the customer there and they choose a coin and network on our crypto checkout. We confirm the transfer on-chain, credit your USD wallet and send the usual webhook with `"gateway": "crypto"`.

USD is the natural currency for crypto. USD `payment_url` also offers card and Apple Pay, so only use `crypto_payment_url` when you want to skip straight to crypto.

Coin	Networks
USDT	TRON (TRC-20), Ethereum (ERC-20), BSC, Polygon
USDC	Ethereum, Polygon, BSC
BTC	Bitcoin
ETH	Ethereum
BNB, SOL, TRX, LTC, DOGE, TON	Their native networks

Currencies and methods

This table is generated live from our payment configuration, so it always shows what's navigable right now. Send the code in the `currency` field





Kenyan Shilling	KE S	M-Pesa prompt, or hosted checkout for Airtel Money, Telkom, Equitel and card	KES 1
Nigerian Naira	NG N	Card, bank transfer, OPay (hosted checkout)	NGN 100
Ghanaian Cedi	GH S	Mobile money	GHS 1
Tanzanian Shilling	TZ S	Mobile money	TZS 500
West African CFA Franc	XO F	Mobile money: Orange, MTN, Wave, Free	XOF 100
US Dollar	US D	Card, Apple Pay, Visa QR, or crypto	USD 0.50
Rwandan Franc	RW F	Mobile money: MTN, Airtel	Set by the network
Ugandan Shilling	UG X	Mobile money: MTN, Airtel	Set by the network
Zambian Kwacha	ZM W	Mobile money: Airtel, MTN, Zamtel	Set by the network
Malawian Kwacha	MW K	Mobile money: Airtel, TNM	Set by the network
Sierra Leonean Leone	SL L	Mobile money: Orange. Show amounts to people as SLE	Set by the network
Congolese Franc	CD F	Mobile money: Airtel, MTN, Orange	Set by the network
Mozambican Metical	MZ N	Mobile money	Set by the network

Central African CFA
Franc

XA
F

Mobile money: MTN, Orange

Set by the
network

Coming soon: ZAR (South African Rand), GNF (Guinean Franc), BIF (Burundian Franc), GMD (Gambian Dalasi), SZL (Eswatini Lilangeni), LRD (Liberian Dollar).

Requests in these currencies return 400 with a "coming soon" message.

🕒 **Sierra Leone:** send SLL in API requests, but label amounts **SLE** when you show them to people. The currency was redenominated in 2022, and SLE is rejected as a request value.

Events and payload

When a payment changes state we send an HTTP POST with a JSON body to the `callback_url` you gave when creating it.

Event	When
<code>payment.succes</code> s	The customer paid. Fulfil the order
<code>payment.fail</code> ed	The payment was declined or timed out
<code>payment.canc</code> elled	The customer cancelled
<code>payment.refund</code> ed	You recorded a refund . <code>reference</code> is the refund's, and <code>metadata.original_reference</code> is the payment's

Handling webhooks well

- **Reply 200 fast**, within 10 seconds, then do the slow work. Slow replies are treated as failures and retried.
- **Expect duplicates.** Retries and resends deliver the same event again, so check whether the order is already paid before fulfilling.

- **Check the amount and currency** against your order before shipping.
- **Don't redirect** the webhook URL. We don't follow redirects, for security.

`gateway` tells you the method type: `mobile_money`, `card_bank`, `crypto`, `manual` or `test`.

Request headers

Copy

```
POST /webhooks/payza HTTP/1.1
Content-Type: application/json
X-Payza-Signature: 5f2b9c...e41a
User-Agent: Payza-Webhook/1.1 (+https://payzaapi.co.ke/docs)
```

Webhook body

Copy

```
{
  "event": "payment.success",
  "reference": "ORDER_1001",
  "status": "success",
  "amount": 2450,
  "fee": 36.75,
  "currency": "KES",
  "gateway": "mobile_money",
  "customer": {
    "email": "jane@example.com",
    "name": "Jane Wanjiku"
  },
  "metadata": {
    "order_id": "1001"
  },
  "paid_at": "2026-09-24T10:02:41+03:00"
}
```

Verify signatures

Every webhook carries an `X-Payza-Signature` header: an HMAC-SHA256 of the **raw request body**, hex-encoded, using your account's **Webhook Signing Secret**.

Find your secret in [Dashboard → API keys → Webhook Signing Secret](#), along with ready-to-copy code and your recent deliveries. It's unique to your account and separate from your API keys. If you regenerate it, update your server at the same time.

- 1 Read the raw body before parsing JSON. Re-encoding changes the bytes and breaks the signature.
- 2 Compute the HMAC with your secret and compare it to the header with a constant-time comparison.
- 3 If they don't match, reply `401` and ignore the request.

Webhook handler PHP Node.js Python Copy

```
<?php
$raw      = file_get_contents('php://input');
$received = $_SERVER['HTTP_X_PAYZA_SIGNATURE'] ?? '';
$expected = hash_hmac('sha256', $raw, getenv('PAYZA_WEBHOOK_SECRET'));

if (!hash_equals($expected, $received)) {
    http_response_code(401);
    exit;
}

$event = json_decode($raw, true);

if ($event['event'] === 'payment.success') {
    // Skip if this order is already marked paid (webhooks can repeat)
    markOrderPaid($event['reference'], $event['amount'], $event['currency']);
}

http_response_code(200);
echo 'OK';
```

Retries

If your endpoint can't be reached, takes longer than 10 seconds, or replies `408`, `429` or any `5xx`, we send the identical payload and signature again on this schedule:

Attempt	1	2	3	4	5	6
Sent after	Immediately	1 min	5 min	15 min	1 hour	6 hours

Other `4xx` replies (such as `401` or `404`) mean the endpoint is misconfigured, so they aren't retried. Fix it, then [resend](#) from the log. If an endpoint fails several times in a row, we queue new events for retry rather than waiting on it.

Delivery log and resend

`GET` `/api/v1/webhooks`

Lists every delivery attempt, newest first. Pass `id` to get one entry in full, including the payload we sent and your server's response.

Query parameters

`id` integer **Optional**

Return a single log entry.

`reference` string **Optional**

Only deliveries for this payment reference.

`event` string **Optional**

Filter by event, e.g. `payment.success`.

`status` string **Optional**

`success` or `failed`.

`page`, `per_page` integer **Optional**

See [pagination](#).

`POST` `/api/v1/webhooks-resend`

Sends a logged webhook again, with the original payload and signature. Handy after fixing your endpoint.

Body parameters

id integer **Required**

The log entry `id` to resend.

requeue boolean **Optional**

Default `true`: if this resend fails too, it rejoins the retry schedule.

GET /api/v1/webhooks

cURL

PHP

Node.js

Python

Copy

```
curl "https://payzaapi.co.ke/api/v1/webhooks?status=failed" \
-H "X-Public-Key: pk_test_XXXXXXX" \
-H "X-Secret-Key: sk_test_XXXXXXX"
```

Response 200 OK

Copy

```
{
  "success": true,
  "webhooks": [
    {
      "id": 812,
      "transaction_reference": "ORDER_1001",
      "event": "payment.success",
      "url": "https://yourshop.co.ke/webhooks/payza",
      "http_status": 500,
      "status": "failed",
      "attempts": 3,
      "last_attempted_at": "2026-09-24 10:18:02",
      "created_at": "2026-09-24 10:02:42"
    }
  ],
  "pagination": {
    "page": 1,
    "per_page": 20,
    "total": 1,
    "total_pages": 1
  }
}
```

```
}  
}
```

POST /api/v1/webhooks-resend

[cURL](#)

[PHP](#)

[Node.js](#)

[Python](#)

[Copy](#)

```
curl -X POST https://payzaapi.co.ke/api/v1/webhooks-resend \  
-H "X-Public-Key: pk_test_XXXXXXX" \  
-H "X-Secret-Key: sk_test_XXXXXXX" \  
-H "Content-Type: application/json" \  
-d '{  
  "id": 812  
}'
```

Payouts

POST /api/v1/payout

Withdraws money from your wallet to a mobile money account, bank account or crypto wallet. The amount is taken from your balance straight away and the payout starts as `pending` while we process it.

Use [payout methods](#) to find valid `method` values, fees and minimums for each currency.

Body parameters

amount number **Required**

Amount to withdraw, before fees. Must be at least the currency's minimum and no more than your balance.

currency string **Optional**

Wallet to withdraw from. Default `KES`.

method string **Required**

A method for that currency, e.g. `mpesa`, `airtel_money`, `bank_transfer`, `mobile_money`, `opay`.

`account_number` string **Required**

Phone number, bank account number or wallet address.

`account_name` string **Required**

Name on the account.

`bank_code` string **Required for bank_transfer**

From [banks](#).

`bank_name` string **Optional**

Bank name, for your records.

GET `/api/v1/payouts`

Lists your payouts. Pass `reference` to get one, including any note from our team.

Query parameters

`reference` string **Optional**

Return one payout.

`status` string **Optional**

`pending`, `processing`, `approved`, `completed` or `rejected`.

`currency` string **Optional**

Filter by currency.

POST `/api/v1/payout`

cURL

PHP

Node.js

Python

Copy

```
curl -X POST https://payzaapi.co.ke/api/v1/payout \
  -H "X-Public-Key: pk_test_xxxxxxx" \
  -H "X-Secret-Key: sk_test_xxxxxxx" \
  -H "Content-Type: application/json" \
  -d '{
```

```
"amount": 20000,
"currency": "KES",
"method": "mpesa",
"account_number": "254712345678",
"account_name": "Mama Njeri Shop"
}'
```

Response 201 Created

Copy

```
{
  "success": true,
  "payout": {
    "reference": "WDR_8F2A91C3B0D4E6A1_1790224410",
    "amount": 20000,
    "fee": 155,
    "net_amount": 19845,
    "currency": "KES",
    "method": "mpesa",
    "status": "pending"
  }
}
```

Payout methods

GET /api/v1/payout-methods

Returns the payout methods, fee and minimum for each currency. Call it when building your withdrawal form instead of hard-coding values, since they can change.

Fees are either `flat`, or `percent` with a `floor` (the fee is whichever is larger). `requires_bank_fields` tells you to collect `bank_code`.

Query parameters

currency string Optional

Return one currency. Omit for all.

GET /api/v1/payout-methods

[cURL](#)

[PHP](#)

[Node.js](#)

[Python](#)

[Copy](#)

```
curl "https://payzaapi.co.ke/api/v1/payout-methods?currency=KES" \  
-H "X-Public-Key: pk_test_xxxxxxx" \  
-H "X-Secret-Key: sk_test_xxxxxxx"
```

Response 200 OK

[Copy](#)

```
{  
  "success": true,  
  "payout_methods": {  
    "currency": "KES",  
    "available": true,  
    "minimum_withdrawal": 500,  
    "fee": {  
      "type": "flat",  
      "amount": 155  
    },  
    "methods": [  
      {  
        "value": "mpesa",  
        "label": "M-Pesa (Safaricom)",  
        "requires_bank_fields": false  
      },  
      {  
        "value": "airtel_money",  
        "label": "Airtel Money",  
        "requires_bank_fields": false  
      },  
      {  
        "value": "bank_transfer",  
        "label": "Bank Transfer",  
        "requires_bank_fields": true  
      }  
    ]  
  }  
}
```

Banks

GET /api/v1/banks

Returns banks and their codes for a country, for `bank_transfer` payouts. The list refreshes daily.

Query parameters

country string Optional

Two-letter country code, e.g. `KE` or `NG`. Default `KE`.

GET /api/v1/banks

[cURL](#)

[PHP](#)

[Node.js](#)

[Python](#)

[Copy](#)

```
curl "https://payzaapi.co.ke/api/v1/banks?country=NG" \  
-H "X-Public-Key: pk_test_xxxxxxx" \  
-H "X-Secret-Key: sk_test_xxxxxxx"
```

Response 200 OK

[Copy](#)

```
{  
  "success": true,  
  "banks": [  
    {  
      "code": "044",  
      "name": "Access Bank"  
    },  
    {  
      "code": "058",  
      "name": "GTBank"  
    }  
  ],  
  "source": "live",  
  "cached": true  
}
```

Refunds

POST /api/v1/refund

Records a full or partial refund against a successful payment. The amount is deducted from your wallet, any linked store order is marked refunded, and a `payment.refunded` webhook goes to the payment's `callback_url`.



Refunds don't send money back to the customer automatically yet. They record the refund and adjust your balance. Return the money to the customer yourself, for example with a [payout](#) to their number.

Body parameters

`reference` string **Required**

Reference of the successful payment.

`amount` number **Optional**

Partial refund amount. Omit to refund everything that's left. You can refund up to the payment's net amount in total.

`reason` string **Optional**

Why you refunded. Saved with the refund.

GET /api/v1/refunds

Lists your refunds. Filter with `reference` (the original payment's).

POST /api/v1/refund

cURL

PHP

Node.js

Python

Copy

```
curl -X POST https://payzaapi.co.ke/api/v1/refund \
-H "X-Public-Key: pk_test_XXXXXXX" \
-H "X-Secret-Key: sk_test_XXXXXXX" \
-H "Content-Type: application/json" \
-d '{
  "reference": "ORDER_1001",
  "amount": 1000,
  "reason": "Item out of stock"
}'
```

Response 201 Created

Copy

```
{
  "success": true,
  "refund": {
    "reference": "RFD_3C9D0E71A5B2F840_1790225102",
    "original_reference": "ORDER_1001",
    "amount": 1000,
    "currency": "KES",
    "status": "success",
    "remaining_refundable": 1413.25
  }
}
```

Customers

POST /api/v1/customers

Creates a customer, or updates the existing one with the same email. `created` in the response tells you which happened.

Body parameters

name string **Required**

Customer's name.

email string **Optional**

Used to match an existing customer.

phone string **Optional**

Phone number.

address string **Optional**

Postal or delivery address.

notes string **Optional**

Internal notes.

GET /api/v1/customers

Lists customers with their order count, total spent and loyalty points. Search with `q` (name, email or phone), or pass `id` for one customer.

POST /api/v1/customers

cURL

PHP

Node.js

Python

Copy

```
curl -X POST https://payzaapi.co.ke/api/v1/customers \
-H "X-Public-Key: pk_test_XXXXXXX" \
-H "X-Secret-Key: sk_test_XXXXXXX" \
-H "Content-Type: application/json" \
-d '{
  "name": "Jane Wanjiku",
  "email": "jane@example.com",
  "phone": "254712345678"
}'
```

Response 201 Created

Copy

```
{
  "success": true,
  "customer": {
    "id": 342,
    "name": "Jane Wanjiku",
    "email": "jane@example.com",
    "phone": "254712345678",
    "source": "api",
    "loyalty_points": 0,
    "total_orders": 0,
    "total_spent": "0.00"
  },
  "created": true
}
```

Plans

POST /api/v1/plans

A plan is what you charge and how often. Create it once, then [subscribe](#) customers to it. `GET /api/v1/plans` lists your plans.

Body parameters

name string **Required**

Shown to customers, e.g. "Gym membership".

amount number **Required**

Amount charged each period.

currency string **Optional**

Default `KES`.

interval string **Optional**

`daily`, `weekly`, `monthly` (default) or `yearly`.

POST /api/v1/plans

cURL

PHP

Node.js

Python

Copy

```
curl -X POST https://payzaapi.co.ke/api/v1/plans \
-H "X-Public-Key: pk_test_XXXXXXX" \
-H "X-Secret-Key: sk_test_XXXXXXX" \
-H "Content-Type: application/json" \
-d '{
  "name": "Gym membership",
  "amount": 3000,
  "currency": "KES",
  "interval": "monthly"
}'
```

Response 201 Created

Copy

```
{
  "success": true,
  "plan": {
    "id": 17,
    "name": "Gym membership",
```

```
{
  "amount": "3000.00",
  "currency": "KES",
  "interval": "monthly",
  "status": "active"
}
```

Subscriptions

POST /api/v1/subscriptions

Subscribes a customer to a plan. The first charge happens one interval from now, then repeats.

- **KES plans** send an M-Pesa prompt to `customer_phone` on each billing date, which the customer approves.
- **USD plans** charge the customer's saved card automatically. The customer must first complete one USD card payment through `POST /api/v1/pay` with the same email, which saves the card.

Body parameters

`plan_id` integer **Required**

An active plan.

`customer_name` string **Required**

Subscriber's name.

`customer_phone` string **Required for KES**

Receives the M-Pesa prompt each period.

`customer_email` string **Required for USD**

Used to find the saved card.

`customer_id` integer **Optional**

Link to a [customer](#) record.

GET /api/v1/subscriptions

Lists subscriptions. Filter by `status` (`active` , `paused` , `cancelled` , `past_due`) or `plan_id` .

POST /api/v1/subscription-cancel

Body parameters

`id` integer **Required**

Subscription ID.

`action` string **Optional**

`cancel` (default) or `pause` .

POST /api/v1/subscriptions

cURLPHPNode.jsPython

Copy

```
curl -X POST https://payzaapi.co.ke/api/v1/subscriptions \
-H "X-Public-Key: pk_test_XXXXXXX" \
-H "X-Secret-Key: sk_test_XXXXXXX" \
-H "Content-Type: application/json" \
-d '{
  "plan_id": 17,
  "customer_name": "Jane Wanjiku",
  "customer_phone": "254712345678",
  "customer_email": "jane@example.com"
}'
```

Response 201 Created

Copy

```
{
  "success": true,
  "subscription": {
    "id": 58,
    "plan_id": 17,
```

```
    "customer_name": "Jane Wanjiku",
    "customer_phone": "254712345678",
    "status": "active",
    "next_charge_at": "2026-10-24 10:05:00",
    "failed_attempts": 0,
    "card_on_file": false
  }
}
```

Invoices

POST /api/v1/invoices

Creates an invoice with a public page your client can view and pay. Set `send` to email it immediately; otherwise it's saved as a draft.

Body parameters

`client_name` string **Required**

Who you're billing.

`client_email` string **Required**

Where the invoice is sent.

`items` array **Required**

Line items: `[{ "description", "qty", "unit_price" }]`. At least one.

`currency` string **Optional**

Default `KES`.

`tax_rate` number **Optional**

Percent added to the subtotal, e.g. `16`.

`discount_amount` number **Optional**

Flat amount off the total.

due_date string **Optional**

YYYY-MM-DD .

send boolean **Optional**

true emails it now.

client_phone, client_address, notes, store_id various **Optional**

Extra details shown on the invoice.

GET /api/v1/invoices

Lists invoices, filterable by status: draft, sent, paid, overdue, cancelled.

POST /api/v1/invoice

Acts on one invoice. GET /api/v1/invoice?invoice_number=... returns it in full.

Body parameters

invoice_number string **Required**

e.g. INV-202609-0142 .

action string **Required**

send, mark_paid or cancel . Paid invoices can't be cancelled.

POST /api/v1/invoices

cURL

PHP

Node.js

Python

Copy

```
curl -X POST https://payzaapi.co.ke/api/v1/invoices \
-H "X-Public-Key: pk_test_XXXXXXX" \
-H "X-Secret-Key: sk_test_XXXXXXX" \
-H "Content-Type: application/json" \
-d '{
  "client_name": "Kamau Hardware",
  "client_email": "accounts@kamauhardware.co.ke",
  "currency": "KES",
  "items": [
    {
```

```
        "description": "Roofing sheets",
        "qty": 12,
        "unit_price": 1200
    },
    {
        "description": "Delivery",
        "qty": 1,
        "unit_price": 1500
    }
],
"tax_rate": 16,
"due_date": "2026-10-15",
"send": true
}'
```

Response 201 Created

Copy

```
{
  "success": true,
  "invoice": {
    "invoice_number": "INV-202609-0142",
    "total": 18444,
    "currency": "KES",
    "status": "sent",
    "public_url": "https://payzaapi.co.ke/dashboard/invoice_view?token=..."
  }
}
```

POS sessions

Drive the Payzaapi point of sale from your own till app or hardware. Sales belong to a session, like a cashier's shift: open one, ring up sales, then close it to reconcile cash.

POST /api/v1/pos/session-open

Body parameters

opening_cash number **Optional**

Cash in the drawer at the start. Default .

store_id integer Optional

Which of your stores this till belongs to.

POST /api/v1/pos/session-close

Body parameters

reference string Required

The session reference.

closing_cash number Optional

Cash counted at the end of the shift.

POST /api/v1/pos/session-open

cURL

PHP

Node.js

Python

Copy

```
curl -X POST https://payzaapi.co.ke/api/v1/pos/session-open \
-H "X-Public-Key: pk_test_XXXXXXX" \
-H "X-Secret-Key: sk_test_XXXXXXX" \
-H "Content-Type: application/json" \
-d '{
  "opening_cash": 2000
}'
```

Response 201 Created

Copy

```
{
  "success": true,
  "session": {
    "id": 91,
    "reference": "SES_7A21C0D4E5F60718_1790222412",
    "store_id": null,
    "opening_cash": "2000.00",
    "closing_cash": null,
    "total_sales": 0,
    "total_revenue": "0.00",
    "status": "open",
  }
}
```

```
    "opened_at": "2026-09-24 08:00:12",  
    "closed_at": null  
  }  
}
```

POS sales

POST /api/v1/pos/sale

Records a completed sale in an open session. Stock goes down automatically for any line with a `product_id`. Prices are before tax, and `tax_rate` is added per line.

Body parameters

`session_reference` string **Required**

From session-open.

`items` array **Required**

```
[{ "product_id"? , "name", "qty", "unit_price", "tax_rate"? }]
```

`payment_method` string **Optional**

`cash` (default), `mobile_money` or `card`.

`cash_received` number **Optional**

For cash sales. Change is calculated for you.

`discount_amount` number **Optional**

Flat amount off the pre-tax subtotal.

`customer_name`, `customer_email`, `customer_phone`, `notes` string **Optional**

Optional details for the receipt.

GET /api/v1/pos/sales

Lists sales. Filter by `session_reference` or `status` (`completed`, `voided`, `refunded`), or pass `reference` for one sale.

POST /api/v1/pos/sale

cURL

PHP

Node.js

Python

Copy

```
curl -X POST https://payzaapi.co.ke/api/v1/pos/sale \
-H "X-Public-Key: pk_test_xxxxxxx" \
-H "X-Secret-Key: sk_test_xxxxxxx" \
-H "Content-Type: application/json" \
-d '{
  "session_reference": "SES_7A21C0D4E5F60718_1790222412",
  "items": [
    {
      "product_id": 204,
      "name": "Bread 400g",
      "qty": 2,
      "unit_price": 65
    },
    {
      "name": "Milk 500ml",
      "qty": 3,
      "unit_price": 60
    }
  ],
  "payment_method": "cash",
  "cash_received": 500
}'
```

Response 201 Created

Copy

```
{
  "success": true,
  "sale": {
    "reference": "POS_9B3E11A0C4D27F85_1790226031",
    "subtotal": 310,
    "tax_amount": 0,
    "discount_amount": 0,
    "total": 310,
    "currency": "KES",
    "payment_method": "cash",
    "change_given": 190,
    "items": [
      "..."
    ]
  }
}
```

```
}
  }
}
```

POS refunds

POST /api/v1/pos/refund

Refunds some or all items from a sale. Items are matched by `product_id`, or by `name` when there's no product ID. You can't refund more than was sold, across all refunds.

Body parameters

`reference` string **Required**

The sale reference.

`items` array **Required**

```
[{ "product_id"?, "name"?, "qty" }]
```

`restock` boolean **Optional**

Put the items back in stock. Default `true`.

`reason` string **Optional**

Why it was refunded.

POST /api/v1/pos/refund

cURL

PHP

Node.js

Python

Copy

```
curl -X POST https://payzaapi.co.ke/api/v1/pos/refund \
-H "X-Public-Key: pk_test_XXXXXXX" \
-H "X-Secret-Key: sk_test_XXXXXXX" \
-H "Content-Type: application/json" \
-d '{
  "reference": "POS_9B3E11A0C4D27F85_1790226031",
  "items": [
    {
```

```
        "product_id": 204,  
        "qty": 1  
      }  
    ],  
    "reason": "Damaged"  
  }'
```

Response 201 Created

Copy

```
{  
  "success": true,  
  "refund": {  
    "sale_reference": "POS_9B3E11A0C4D27F85_1790226031",  
    "amount": 65,  
    "tax_amount": 0,  
    "currency": "KES",  
    "fully_refunded": false,  
    "items": [  
      {  
        "product_id": 204,  
        "name": "Bread 400g",  
        "qty": 1,  
        "amount": 65  
      }  
    ]  
  }  
}
```

Plugins and Postman

Ready-made integrations that use the same public API documented here. Source code is included, so you can read or adapt them.



WooCommerce

Payment gateway for WordPress stores. WordPress 5.8+, WooCommerce 6.0+, PHP 7.4+.

[Download](#)



WHMCS

Gateway module for hosting and billing. Invoices are marked paid automatically. WHMCS 8.0+.

[Download](#)



Postman collection

Every endpoint with example bodies and a test environment. Paste your keys once.

[Download](#)

Install on WooCommerce

1. In WordPress, go to **Plugins** → **Add New** → **Upload Plugin**, choose the zip and activate it.
2. Open **WooCommerce** → **Settings** → **Payments** → **Payza** and paste your keys.
3. Place a test order with test keys. It should move to **Processing** when the webhook arrives. The webhook URL is configured for you.

Install on WHMCS

1. Upload the zip's contents to your WHMCS root so `modules/gateways/payza.php` is in place.
2. In **Configuration** → **Payment Gateways**, activate Payza and enter your keys.
3. Each invoice's `callback_url` is set to `/modules/gateways/callback/payza.php` automatically.

Go-live checklist

1. Your server verifies `X-Payza-Signature` on every webhook and rejects mismatches.
 2. Your webhook handler replies `200` within 10 seconds and ignores events it has already processed.
 3. Orders are fulfilled from the webhook (or [verify](#)), never from the customer's redirect alone.
 4. You check `amount` and `currency` against the order before fulfilling.
 5. Every payment uses a unique `reference`, stored against your order.
 6. Keys live in environment variables on your server, and the secret key is never sent to a browser or app.
 7. Your `callback_url` is HTTPS and doesn't redirect.
 8. You've swapped test keys for `pk_live_` and `sk_live_` keys.
-

FAQ

How do I accept Airtel Money, Telkom or Equitel in Kenya?	+
When should I use <code>stk_push</code> true or false?	+
Does <code>payment_url</code> work for Safaricom customers too?	+
Why do I get 403 for some currencies?	+
How long does a customer have to pay?	+
Can I reuse a reference?	+
My webhook isn't arriving. What should I check?	+

Changelog

Change	Details
Webhook retries	Failed deliveries are retried six times over about 7.5 hours, and can be resent from the delivery log.
Per-account webhook secrets	Each account signs webhooks with its own secret from Dashboard → API keys.
API v2 endpoints	Refunds, payouts, payout methods, banks, customers, plans, subscriptions, invoices, POS and the webhook log.
USD cards	USD <code>payment_url</code> offers card, Apple Pay and Visa QR alongside crypto.
All Kenyan networks	<code>"stk_push": false</code> opens hosted checkout for Airtel Money, Telkom and Equitel.